MODUL PRAKTIK

Pengujian Website Data Anak & EMIS

Dari Unit Test sampai UAT — untuk programmer lokal yang mengerjakan sistem kesehatan & pendidikan anak (puskesmas, guru, laporan nasional).

Stack: **Django + Django REST Framework + PostgreSQL**
7 jenis pengujian • pytest-django, Postman/Newman, Supertest, Selenium, Playwright, k6, OWASP ZAP • template UAT, test case & bug report • pipeline CI • roadmap belajar 8 minggu.

Catatan asumsi. *Nama app, model, URL, dan role pada contoh kode adalah contoh yang masuk akal untuk sistem data anak/EMIS. Sesuaikan dengan kode proyek Anda yang sebenarnya.*

Daftar Isi

Bab 0 — Peta Pengujian & Cara Memakai Modul 5

0.1 Kenapa pengujian jadi kewajiban di proyek ini . . . 5
0.2 Tujuh jenis pengujian sekilas . . . 5
0.3 Tiga yang wajib Anda kuasai (programmer lokal) . . . 5
0.4 Padanan istilah: Laravel → Django . . . 5
0.5 Piramida pengujian: berapa banyak tiap jenis? . . . 6
0.6 Struktur folder yang disarankan . . . 6
0.7 Cara belajar dari modul ini . . . 6

Bab 1 — Menyiapkan Lingkungan Uji 7

1.1 Prasyarat . . . 7
1.2 Paket pengujian . . . 7
1.3 Database khusus uji di PostgreSQL . . . 7
1.4 conftest.py: fixture bersama . . . 8
1.5 Factory: pabrik data palsu . . . 8
1.6 Perintah seed untuk E2E dan UAT . . . 9
1.7 Perintah harian . . . 9

Bab 2 — Unit Testing (pytest-django, Jest, Vitest) 10

2.1 Konsep . . . 10
2.2 Contoh 1 — Fungsi hitung umur anak . . . 10
2.3 Contoh 2 — Fungsi login . . . 11
2.4 Mock dan fake: memutus ketergantungan . . . 11
2.5 Mengukur query: menangkap masalah N+1 . . . 12
2.6 Unit test sisi JavaScript: Jest dan Vitest . . . 12
2.7 Apa yang layak dan tidak layak di-unit-test . . . 13
2.8 Latihan bab 2 . . . 13

Bab 3 — Integration & API Testing (DRF, Postman, Supertest) 14

3.1 Konsep . . . 14
3.2 Kode yang diuji (ringkas) . . . 14
3.3 Test API dengan APIClient . . . 14
3.4 Postman: pengujian API tanpa kode . . . 15
3.5 Supertest: API testing dari sisi Node.js . . . 16
3.6 Kapan memakai yang mana . . . 17
3.7 Latihan bab 3 . . . 17

Bab 4 — Functional Testing (Manual + Selenium) 18

4.1 Konsep . . . 18

4.2 Dari requirement ke test case . . . 18
4.3 Contoh test case manual . . . 18
4.4 Teknik merancang data uji . . . 19
4.5 Selenium: mengotomatiskan test fungsional . . . 19
4.6 Traceability matrix . . . 20
4.7 Latihan bab 4 . . . 20

Bab 5 — End-to-End (E2E) Testing dengan Playwright 21

5.1 Konsep . . . 21
5.2 Cypress vs Playwright vs Selenium . . . 21
5.3 Instalasi dan data uji . . . 21
5.4 Konfigurasi . . . 21
5.5 Login sekali, pakai berkali-kali . . . 22
5.6 Page Object: halaman sebagai kelas . . . 22
5.7 Alur utama: Login → Input anak → Simpan → Laporan muncul . . . 23
5.8 Menjalankan dan men-debug . . . 24
5.9 Praktik pro E2E . . . 24
5.10 Varian Python: pytest-playwright + live_server . . . 24
5.11 Latihan bab 5 . . . 25

Bab 6 — Performance Testing (k6, JMeter) 26

6.1 Konsep . . . 26
6.2 Jenis uji beban . . . 26
6.3 Dari '1000 orang' menjadi angka teknis . . . 26
6.4 Menulis skrip k6 . . . 26
6.5 Membaca hasil . . . 27
6.6 Titik penyempitan umum di Django + PostgreSQL . . . 28
6.7 JMeter: alternatif berbasis GUI . . . 28
6.8 Aturan main uji beban . . . 28
6.9 Latihan bab 6 . . . 28

Bab 7 — Security Testing (OWASP ZAP) — WAJIB untuk Data Anak 29

7.1 Aturan keterlibatan (rules of engagement) . . . 29
7.2 OWASP Top 10 dan padanannya di Django . . . 29
7.3 Pengecekan otomatis yang murah dan efektif . . . 29
7.4 Test keamanan sebagai kode (pytest) . . . 30
7.5 OWASP ZAP: pemindai kerentanan . . . 31
7.6 Triase temuan . . . 31
7.7 Keamanan khusus data anak . . . 32
7.8 Checklist keamanan sebelum go-live . . . 32
7.9 Latihan bab 7 . . . 32

Bab 8 — User Acceptance Testing (UAT) **33**

8.1 Konsep . . . 33
8.2 Syarat masuk UAT (entry criteria) . . . 33
8.3 Rencana UAT . . . 33
8.4 Skrip skenario UAT (bahasa user, bukan bahasa teknis) . . . 33
8.5 Definisi tingkat keparahan temuan . . . 34
8.6 Kriteria lulus (exit criteria) . . . 34
8.7 Formulir penerimaan (sign-off) . . . 34
8.8 Tips mengelola UAT . . . 34

Bab 9 — Menyatukan Semuanya: CI/CD, Strategi, Roadmap **35**

9.1 Pipeline CI (GitHub Actions) . . . 35
9.2 Kapan setiap jenis tes dijalankan . . . 36
9.3 Definition of Done (DoD) untuk setiap fitur . . . 36
9.4 Checklist rilis . . . 36
9.5 Metrik kualitas yang layak dilaporkan . . . 37
9.6 Roadmap belajar 8 minggu untuk programmer lokal . . . 37
9.7 Template dokumen Strategi Pengujian (1-2 halaman) . . . 37

Lampiran **38**

A. Template test case . . . 38
B. Template laporan bug . . . 38
C. Contekan perintah . . . 38
D. Glosarium . . . 39

Bab 0 — Peta Pengujian & Cara Memakai Modul

0.1 Kenapa pengujian jadi kewajiban di proyek ini

Website ini menyimpan **data anak** (identitas, status kesehatan, data sekolah/EMIS) dan dipakai banyak pihak sekaligus: petugas puskesmas, guru, admin kabupaten, dan akhirnya UNICEF atau pemerintah. Satu bug kecil bisa berarti laporan nasional salah, data anak bocor, atau seluruh puskesmas tidak bisa input pada hari yang sama. Karena itu pengujian bukan 'kalau sempat', tetapi bagian dari definisi *selesai* sebuah fitur.

0.2 Tujuh jenis pengujian sekilas

#	Jenis	Pertanyaan yang dijawab	Contoh di proyek	Tools	Siapa
1	Unit	Apakah satu fungsi/kelas benar?	hitung umur anak, validasi NIK, login	pytest-django (setara PHPUnit), Jest, Vitest	Programmer
2	Integration	Apakah dua modul nyambung?	form pendaftaran anak masuk ke database	DRF APIClient, Postman, Supertest	Programmer
3	Functional	Apakah fitur sesuai kebutuhan user?	guru bisa input data EMIS	Manual + Selenium	QA / programmer
4	E2E	Apakah alur lengkap jalan seperti user asli?	Login > input anak > simpan > laporan muncul	Playwright, Cypress, Selenium	QA / programmer
5	Performance	Kuat dibuka 1000 orang sekaligus?	semua puskesmas akses bersamaan	k6, JMeter	Programmer / DevOps
6	Security	Bisakah hacker membobol data anak?	IDOR, SQL injection, XSS, brute force	OWASP ZAP + test manual	Programmer / security
7	UAT	Apakah klien menyatakan 'OK, lulus'?	UNICEF / Pemerintah mencoba sendiri	Skenario + formulir sign-off	Klien

0.3 Tiga yang wajib Anda kuasai (programmer lokal)

Prioritas	Kemampuan	Alat	Bab
1	Unit Testing	pytest-django (untuk Django, ini yang setara PHPUnit di Laravel)	2
2	API Testing	Postman + Newman, ditambah DRF APIClient	3
3	E2E Testing	Playwright — paling dicari sekarang, paling penting untuk UNICEF	5

Empat lainnya (Functional, Performance, Security, UAT) tetap dibahas lengkap karena proyek data anak **pasti** akan ditanyai soal keamanan dan beban, dan UAT adalah gerbang terakhir sebelum go-live.

0.4 Padanan istilah: Laravel → Django

Banyak tutorial testing memakai Laravel. Tabel ini membantu menerjemahkannya ke Django.

Di Laravel	Di Django (modul ini)
PHPUnit	pytest + pytest-django (atau `django.test.TestCase`)
RefreshDatabase	`@pytest.mark.django_db` (rollback otomatis per test)

Di Laravel	Di Django (modul ini)
Model Factory	factory_boy + Faker
actingAs / Sanctum::actingAs	`client.force_login()` / `APIClient.force_authenticate()`
Http::fake	`responses` / `unittest.mock.patch`
Mail::fake	fixture `mailoutbox`
php artisan test	`pytest`
php artisan db:seed	`python manage.py seed_qa` (management command buatan sendiri)

0.5 Piramida pengujian: berapa banyak tiap jenis?

Lapisan	Porsi ideal	Kecepatan	Biaya perawatan	Aturan praktis
Unit	~60-70%	milidetik	rendah	Semua logika bisnis (umur, status gizi, validasi) wajib punya unit test
Integration / API	~20-25%	detik	sedang	Setiap endpoint: sukses, gagal validasi, tanpa login, salah role
E2E	~5-10%	puluhan detik	tinggi	Hanya alur kritis: login, input anak, simpan, laporan, ekspor
Manual / UAT	sisanya	jam-hari	tinggi	Eksplorasi, kenyamanan, bahasa, kasus tak terduga

> **TIPS PRO.** Jangan membalik piramida. Tim yang hanya mengandalkan E2E akan punya test lambat, rapuh, dan akhirnya dimatikan. E2E itu payung terakhir, bukan fondasi.

0.6 Struktur folder yang disarankan

Struktur proyek

```
proyek/
  config/settings/{base,dev,test,e2e,prod}.py
  anak/                  <- app Django: models, serializers, views, services
    tests/
      test_umur.py       <- unit
      test_api_anak.py   <- integration / API
  emis/ laporan/ users/  <- app lain, masing-masing punya tests/
  conftest.py            <- fixture bersama (api_client, petugas, guru, ...)
  pytest.ini
  e2e/                   <- Playwright (TypeScript)
    pages/  tests/  playwright.config.ts
  qa/
    postman/             <- collection + environment
    k6/                  <- skrip beban
    zap/                 <- konfigurasi & laporan scan
    uat/                 <- skenario + formulir sign-off
  .github/workflows/tests.yml
```

0.7 Cara belajar dari modul ini

1. Baca bab teorinya, lalu **ketik sendiri** contoh kodenya (jangan copy-paste) di proyek latihan.
2. Kerjakan bagian *Latihan* di akhir tiap bab.
3. Setelah bab 2, 3, 5 selesai, Anda sudah layak mengaku bisa menguji proyek ini.
4. Bab 6-8 dikerjakan menjelang rilis atau sebelum audit.

Bab 1 — Menyiapkan Lingkungan Uji

Semua pengujian otomatis butuh lingkungan yang **terisolasi**: database khusus uji, data palsu, dan konfigurasi yang tidak menyentuh data asli. Aturan emas: **jangan pernah menguji dengan data anak sungguhan**.

1.1 Prasyarat

Komponen	Versi disarankan	Cek dengan
Python	3.11+	python --version
Django / DRF	4.2 LTS atau 5.x / 3.15+	python -m django --version
PostgreSQL	15 / 16	psql --version
Node.js (untuk Playwright)	20 LTS+	node -v
Docker (opsional)	terbaru	docker -v
Postman + Newman	terbaru	newman -v
k6	terbaru	k6 version

1.2 Paket pengujian

Install: pip install -r requirements-dev.txt

```
# requirements-dev.txt
pytest
pytest-django
pytest-cov            # laporan coverage
pytest-xdist          # jalankan test paralel (-n auto)
factory-boy           # pabrik data palsu
faker
responses             # memalsukan panggilan HTTP keluar
selenium              # bab 4 (opsional)
bandit                # analisis keamanan statis (bab 7)
pip-audit             # cek kerentanan dependensi (bab 7)
```

1.3 Database khusus uji di PostgreSQL

Django otomatis membuat database `test_<nama>` saat test dimulai dan menghapusnya setelah selesai, sehingga database asli aman. Syaratnya: user PostgreSQL punya hak **CREATEDB**. Pakai PostgreSQL asli (bukan SQLite) agar perilaku sama dengan produksi: constraint, collation, JSON field, query agregasi laporan.

Menyiapkan user

```
-- sekali saja, di psql sebagai superuser
CREATE USER anak_test WITH PASSWORD 'ganti-ini' CREATEDB;
CREATE DATABASE anak_emis OWNER anak_test;   -- DB dev; DB uji dibuat otomatis oleh Django
```

settings khusus test

```
# config/settings/test.py
from .base import *

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ.get("TEST_DB_NAME", "anak_emis"),   # Django akan memakai test_anak_emis
        "USER": os.environ.get("TEST_DB_USER", "anak_test"),
        "PASSWORD": os.environ.get("TEST_DB_PASSWORD", "ganti-ini"),
        "HOST": os.environ.get("TEST_DB_HOST", "127.0.0.1"),
        "PORT": os.environ.get("TEST_DB_PORT", "5432"),
    }
}
```

```
PASSWORD_HASHERS = ["django.contrib.auth.hashers.MD5PasswordHasher"]   # hash cepat, HANYA untuk test
EMAIL_BACKEND = "django.core.mail.backends.locmem.EmailBackend"
CELERY_TASK_ALWAYS_EAGER = True            # jika memakai Celery
DEBUG = False
```

pytest.ini

```
# pytest.ini
[pytest]
DJANGO_SETTINGS_MODULE = config.settings.test
python_files = tests.py test_*.py *_tests.py
addopts = -ra --strict-markers --reuse-db
markers =
    slow: test lambat (jalankan terpisah)
```

> **TIPS PRO.** `--reuse-db` mempercepat run berikutnya karena database uji tidak dibuat ulang. Setelah mengubah model/migrasi, jalankan sekali dengan `pytest --create-db`.

1.4 conftest.py: fixture bersama

Fixture

```
# conftest.py (di root proyek)
import pytest
from rest_framework.test import APIClient
from users.tests.factories import UserFactory, PuskesmasFactory


@pytest.fixture
def api_client():
    return APIClient()


@pytest.fixture
def puskesmas(db):
    return PuskesmasFactory()


@pytest.fixture
def petugas(puskesmas):
    return UserFactory(role="petugas_puskesmas", puskesmas=puskesmas)


@pytest.fixture
def guru(db):
    return UserFactory(role="guru", puskesmas=None)


@pytest.fixture
def petugas_client(api_client, petugas):
    api_client.force_authenticate(petugas)     # lewati login, langsung terautentikasi
    return api_client
```

1.5 Factory: pabrik data palsu

factory_boy membuat objek model realistis tanpa mengetik manual. Untuk data anak gunakan nama dan NIK **fiktif**.

Factory

```
# users/tests/factories.py
import factory
from factory.django import DjangoModelFactory
from anak.models import Anak, Puskesmas


class PuskesmasFactory(DjangoModelFactory):
    class Meta:
        model = Puskesmas

    kode = factory.Sequence(lambda n: f"PKM-{n:04d}")
    nama = factory.Faker("company")
```

```
class UserFactory(DjangoModelFactory):
    class Meta:
        model = "users.User"          # model User kustom proyek Anda

    username = factory.Sequence(lambda n: f"user{n}")
    role = "petugas_puskesmas"
    puskesmas = factory.SubFactory(PuskesmasFactory)
    password = factory.PostGenerationMethodCall("set_password", "Rahasia-123")


class AnakFactory(DjangoModelFactory):
    class Meta:
        model = Anak

    nama = factory.Faker("name")
    nik = factory.Sequence(lambda n: f"53010101{n:08d}")        # 16 digit, unik
    jenis_kelamin = factory.Iterator(["L", "P"])
    tanggal_lahir = factory.Faker("date_between", start_date="-12y", end_date="-2m")
    nama_ibu = factory.Faker("name_female")
    puskesmas = factory.SubFactory(PuskesmasFactory)
```

1.6 Perintah seed untuk E2E dan UAT

E2E dan UAT butuh data dasar yang **tetap**: satu akun per role, satu puskesmas, satu sekolah. Buat *management command* agar bisa dijalankan ulang kapan saja.

seed_qa

```
# users/management/commands/seed_qa.py   ->   python manage.py seed_qa
import os
from django.contrib.auth import get_user_model
from django.core.management.base import BaseCommand, CommandError
from anak.models import Puskesmas


class Command(BaseCommand):
    help = "Isi data dasar QA (HANYA lingkungan non-produksi)"

    def handle(self, *args, **opts):
        if os.environ.get("DJANGO_ENV") == "production":
            raise CommandError("Dilarang dijalankan di produksi.")
        User = get_user_model()
        pkm, _ = Puskesmas.objects.get_or_create(kode="PKM-QA-01", defaults={"nama": "Puskesmas QA Dili"})
        for username, role in [("petugas_qa", "petugas_puskesmas"), ("guru_qa", "guru"),
                               ("admin_qa", "admin_kabupaten")]:
            u, _ = User.objects.get_or_create(username=username, defaults={"role": role, "puskesmas": pkm})
            u.set_password("Rahasia-QA-123")
            u.save()
        self.stdout.write(self.style.SUCCESS("Data QA siap."))
```

> **WAJIB / BAHAYA.** Guard `DJANGO_ENV == production` di atas wajib. Satu kesalahan menjalankan seed dengan password lemah di produksi adalah celah keamanan.

1.7 Perintah harian

Tujuan	Perintah
Jalankan semua test	pytest
Berhenti di kegagalan pertama	pytest -x
Satu file / satu test	pytest anak/tests/test_api_anak.py -k tambah_anak
Paralel (lebih cepat)	pytest -n auto
Dengan coverage	pytest --cov=anak --cov-report=term-missing --cov-fail-under=70
Setelah ubah migrasi	pytest --create-db
Cek konfigurasi Django	python manage.py check

Bab 2 — Unit Testing (pytest-django, Jest, Vitest)

2.1 Konsep

Unit test menguji **satu unit kecil** (fungsi/kelas) secara terisolasi: tanpa jaringan, idealnya tanpa database. Hasilnya cepat (ratusan test dalam hitungan detik) sehingga bisa dijalankan setiap kali menyimpan file.

Pola AAA: *Arrange* (siapkan data) → *Act* (jalankan fungsi) → *Assert* (periksa hasil). Satu test = satu perilaku. Nama test harus bercerita: `test_tanggal_lahir_masa_depan_ditolak`.

2.2 Contoh 1 — Fungsi hitung umur anak

Logika umur dan kategori (bayi, baduta, balita) dipakai di hampir semua laporan. Salah satu bulan saja membuat anak masuk kategori yang salah, jadi ini **kandidat unit test nomor satu**. Karena fungsi murni, tidak butuh database.

Kode yang diuji

```
# anak/services/umur.py
from datetime import date


def hitung_bulan(lahir: date, acuan: date) -> int:
    if lahir > acuan:
        raise ValueError("Tanggal lahir tidak boleh di masa depan.")
    bulan = (acuan.year - lahir.year) * 12 + (acuan.month - lahir.month)
    if acuan.day < lahir.day:          # bulan berjalan belum genap
        bulan -= 1
    return bulan


def kategori(bulan: int) -> str:
    if bulan < 12:
        return "bayi"
    if bulan < 24:
        return "baduta"
    if bulan < 60:
        return "balita"
    return "anak"
```

Test-nya

```
# anak/tests/test_umur.py
from datetime import date
import pytest
from anak.services.umur import hitung_bulan, kategori


def test_tepat_satu_tahun_adalah_12_bulan():
    assert hitung_bulan(date(2024, 3, 15), date(2025, 3, 15)) == 12


def test_sehari_sebelum_ulang_bulan_belum_genap():
    assert hitung_bulan(date(2024, 3, 15), date(2025, 3, 14)) == 11


def test_tanggal_lahir_masa_depan_ditolak():
    with pytest.raises(ValueError, match="masa depan"):
        hitung_bulan(date(2030, 1, 1), date(2025, 1, 1))


@pytest.mark.parametrize("bulan, harapan", [      # sengaja menguji NILAI BATAS (boundary)
    (0, "bayi"), (11, "bayi"),
    (12, "baduta"), (23, "baduta"),
    (24, "balita"), (59, "balita"),
    (60, "anak"),
])
```

```
def test_kategori_umur(bulan, harapan):
    assert kategori(bulan) == harapan
```

> **TIPS PRO.** Teknik **boundary value** (uji 11/12, 23/24, 59/60) menangkap bug off-by-one yang paling sering lolos. Untuk setiap aturan angka, uji di batas, tepat di bawah, dan tepat di atas.

2.3 Contoh 2 — Fungsi login

Login menyentuh database dan session, jadi diuji dengan `@pytest.mark.django_db` (setiap test dibungkus transaksi yang di-rollback otomatis). Yang diuji: sukses, password salah, akun nonaktif, dan halaman terlindungi.

Test login

```
# users/tests/test_login.py
import pytest
from django.urls import reverse

pytestmark = pytest.mark.django_db


def test_login_berhasil_dan_masuk_dashboard(client, django_user_model):
    django_user_model.objects.create_user(username="petugas1", password="Rahasia-123")

    resp = client.post(reverse("login"), {"username": "petugas1", "password": "Rahasia-123"})

    assert resp.status_code == 302
    assert resp.url == "/dashboard/"
    assert "_auth_user_id" in client.session          # sesi terbentuk


def test_password_salah_ditolak(client, django_user_model):
    django_user_model.objects.create_user(username="petugas1", password="Rahasia-123")

    resp = client.post(reverse("login"), {"username": "petugas1", "password": "salah"})

    assert resp.status_code == 200                     # form tampil ulang dengan error
    assert "_auth_user_id" not in client.session


def test_akun_nonaktif_tidak_bisa_login(client, django_user_model):
    django_user_model.objects.create_user(username="x", password="Rahasia-123", is_active=False)

    client.post(reverse("login"), {"username": "x", "password": "Rahasia-123"})

    assert "_auth_user_id" not in client.session


def test_dashboard_butuh_login(client):
    resp = client.get("/dashboard/")
    assert resp.status_code == 302
    assert resp.url.startswith("/accounts/login/")      # sesuaikan dengan LOGIN_URL
```

2.4 Mock dan fake: memutus ketergantungan

Kalau fungsi memanggil layanan luar (SMS, email, API EMIS pemerintah), jangan panggil yang asli saat test. Gunakan `responses` untuk HTTP dan fixture `mailoutbox` untuk email.

Mock HTTP dan email

```
import responses
from anak.services.emis import kirim_ke_emis
from users.tests.factories import AnakFactory


@responses.activate
@pytest.mark.django_db
def test_sinkron_emis_gagal_tidak_crash():
    responses.add(responses.POST, "https://emis.example.gov/api/siswa", status=500)
    anak = AnakFactory()
```

```
    hasil = kirim_ke_emis(anak)

    assert hasil.sukses is False                        # melapor gagal, bukan exception
    assert len(responses.calls) == 1


@pytest.mark.django_db
def test_pendaftaran_mengirim_email_notifikasi(mailoutbox):
    # ... jalankan fungsi pendaftaran ...
    assert len(mailoutbox) == 1
    assert "Pendaftaran" in mailoutbox[0].subject
```

2.5 Mengukur query: menangkap masalah N+1

Laporan yang lambat biasanya karena N+1 query. pytest-django menyediakan fixture `django_assert_max_num_queries` sebagai pagar pengaman.

Pagar query

```
@pytest.mark.django_db
def test_daftar_anak_tidak_n_plus_1(petugas_client, petugas, django_assert_max_num_queries):
    AnakFactory.create_batch(50, puskesmas=petugas.puskesmas)

    with django_assert_max_num_queries(6):               # tetap kecil walau data 50 atau 5000
        resp = petugas_client.get("/api/anak/")

    assert resp.status_code == 200
```

2.6 Unit test sisi JavaScript: Jest dan Vitest

Jika front-end punya logika JS sendiri (React/Vue atau skrip validasi form), logika murninya diuji dengan Jest (proyek Node lama) atau Vitest (proyek Vite). Sintaksnya hampir sama. Jika halaman hanya template Django tanpa logika JS berarti, bagian ini boleh dilewati.

Vitest / Jest

```
// src/utils/umur.js
export function umurBulan(lahir, acuan = new Date()) {
  const l = new Date(lahir);
  if (l > acuan) throw new Error('Tanggal lahir tidak boleh di masa depan');
  let bulan = (acuan.getFullYear() - l.getFullYear()) * 12 + (acuan.getMonth() - l.getMonth());
  if (acuan.getDate() < l.getDate()) bulan -= 1;
  return bulan;
}

// src/utils/umur.test.js   (Vitest; untuk Jest hapus baris import vitest)
import { describe, it, expect } from 'vitest';
import { umurBulan } from './umur';

describe('umurBulan', () => {
  it('12 bulan pas satu tahun', () => {
    expect(umurBulan('2024-03-15', new Date('2025-03-15'))).toBe(12);
  });
  it('belum genap jika tanggal belum lewat', () => {
    expect(umurBulan('2024-03-15', new Date('2025-03-14'))).toBe(11);
  });
  it.each([['2030-01-01'], ['2099-12-31']])('menolak masa depan: %s', (tgl) => {
    expect(() => umurBulan(tgl, new Date('2025-01-01'))).toThrow();
  });
});
```

Menjalankan

```
npm i -D vitest            # atau: npm i -D jest
npx vitest run             # sekali jalan (CI)
npx vitest                 # mode watch saat coding
npx vitest run --coverage  # laporan coverage
```

2.7 Apa yang layak dan tidak layak di-unit-test

Layak (prioritas tinggi)	Tidak perlu
Hitung umur, kategori usia, status gizi, skor	Getter/setter sepele
Validasi NIK, nomor telepon, format tanggal (validator & serializer)	Kode framework (sudah diuji Django)
Aturan otorisasi (permission class)	Template murni tanpa logika
Query manager/queryset & agregasi laporan	Konfigurasi statis
Penanganan error, nilai batas, nilai kosong	Duplikasi test yang sama di banyak tempat

2.8 Latihan bab 2

1. Tambahkan `status_gizi(berat_kg, tinggi_cm, umur_bulan)` sederhana dan uji 6 kasus termasuk nilai batas.
2. Buat test validator NIK 16 digit: tolak 15 digit, huruf, spasi, dan string kosong.
3. Ukur coverage `anak/services`; targetkan di atas 80%.
4. Sengaja rusak satu baris (ubah < jadi <=). Pastikan test Anda gagal — itu bukti test Anda berguna.

Bab 3 — Integration & API Testing (DRF, Postman, Supertest)

3.1 Konsep

Integration test memeriksa bahwa **dua atau lebih bagian bekerja bersama**: view + serializer + permission + database PostgreSQL. Pertanyaan khasnya: *"Apakah form pendaftaran anak benar-benar masuk ke database, dan hanya boleh dilakukan oleh petugas yang berhak?"*

Untuk setiap endpoint, gunakan **checklist 6 skenario**:

#	Skenario	Hasil yang diharapkan (DRF)
1	Sukses (data valid, role benar)	200/201 + data tersimpan di DB
2	Data tidak valid (kosong, format salah, terlalu panjang)	400 + pesan error per field
3	Tanpa login	401 (token/JWT) atau 403 (hanya SessionAuthentication)
4	Login tapi role salah	403
5	Mengakses data milik puskesmas lain	404 (queryset difilter) atau 403; tidak boleh bocor
6	Duplikat (NIK sama)	400 (UniqueValidator), tidak ada baris ganda

3.2 Kode yang diuji (ringkas)

Perhatikan dua hal penting: **puskesmas diambil dari akun**, bukan dari input; dan **queryset difilter** per puskesmas.

Serializer dan ViewSet

```
# anak/serializers.py
class AnakSerializer(serializers.ModelSerializer):
    class Meta:
        model = Anak
        fields = ["id", "nama", "nik", "jenis_kelamin", "tanggal_lahir", "nama_ibu", "puskesmas"]
        read_only_fields = ["id", "puskesmas"]          # klien TIDAK boleh menentukan puskesmas

    def validate_tanggal_lahir(self, v):
        if v > date.today():
            raise serializers.ValidationError("Tanggal lahir tidak boleh di masa depan.")
        return v

# anak/views.py
class AnakViewSet(viewsets.ModelViewSet):
    serializer_class = AnakSerializer
    permission_classes = [IsAuthenticated, IsPetugasPuskesmas]   # role lain -> 403

    def get_queryset(self):
        return Anak.objects.filter(puskesmas=self.request.user.puskesmas)   # anti-IDOR

    def perform_create(self, serializer):
        serializer.save(puskesmas=self.request.user.puskesmas)
```

3.3 Test API dengan APIClient

Integration test anak

```
# anak/tests/test_api_anak.py
import pytest
from anak.models import Anak
from users.tests.factories import AnakFactory, UserFactory

pytestmark = pytest.mark.django_db
```

```
URL = "/api/anak/"


def payload(**override):
    data = {"nama": "Maria da Silva", "nik": "5301010101200001", "jenis_kelamin": "P",
            "tanggal_lahir": "2022-05-10", "nama_ibu": "Ana da Silva"}
    data.update(override)
    return data


def test_petugas_bisa_tambah_anak_dan_tersimpan_di_database(petugas_client, petugas):
    resp = petugas_client.post(URL, payload(), format="json")

    assert resp.status_code == 201
    assert resp.json()["nama"] == "Maria da Silva"
    anak = Anak.objects.get(nik="5301010101200001")
    assert anak.puskesmas_id == petugas.puskesmas_id        # otomatis dari akun


def test_data_tidak_valid_ditolak_400(petugas_client):
    resp = petugas_client.post(URL, payload(nik="123", tanggal_lahir="2999-01-01"), format="json")

    assert resp.status_code == 400
    assert {"nik", "tanggal_lahir"} <= set(resp.json())     # error per field
    assert Anak.objects.count() == 0


def test_tanpa_login_ditolak(api_client):
    resp = api_client.post(URL, payload(), format="json")
    assert resp.status_code in (401, 403)


def test_role_guru_tidak_boleh_input_data_puskesmas(api_client, guru):
    api_client.force_authenticate(guru)
    assert api_client.post(URL, payload(), format="json").status_code == 403


def test_nik_duplikat_ditolak(petugas_client):
    AnakFactory(nik="5301010101200001")

    resp = petugas_client.post(URL, payload(), format="json")

    assert resp.status_code == 400
    assert Anak.objects.count() == 1


def test_petugas_tidak_bisa_melihat_anak_puskesmas_lain(petugas_client):   # anti-IDOR
    anak_lain = AnakFactory()                       # puskesmas berbeda

    resp = petugas_client.get(f"{URL}{anak_lain.id}/")

    assert resp.status_code == 404                  # tidak mengaku bahwa datanya ada


def test_klien_tidak_bisa_memaksa_puskesmas_lain(petugas_client, petugas):   # mass assignment
    lain = UserFactory().puskesmas
    petugas_client.post(URL, payload(puskesmas=lain.id), format="json")

    assert Anak.objects.get().puskesmas_id == petugas.puskesmas_id
```

PERHATIAN. Dua test terakhir adalah yang paling bernilai untuk data anak: mereka membuktikan data satu puskesmas tidak bocor ke puskesmas lain, meski penyerang mengubah ID di URL atau menambah field palsu di body.

3.4 Postman: pengujian API tanpa kode

Postman cocok untuk eksplorasi cepat, dokumentasi API, dan berbagi koleksi test dengan rekan atau QA. Langkah kerja yang rapi:

1. Buat **Collection** 'API Data Anak' dengan folder per modul (Auth, Anak, EMIS, Laporan).

2. Buat **Environment** terpisah: *local*, *staging*. Isi variabel `base_url`, `username`, `password`. Jangan simpan password produksi.
3. Request 'Login' menyimpan token otomatis ke variabel (skrip di bawah).
4. Setiap request punya tab **Tests** berisi pengecekan otomatis.
5. Jalankan seluruh koleksi dengan **Collection Runner** atau **Newman** (CLI) di CI.

TIPS PRO. Untuk Postman, pakai autentikasi **token/JWT** (header Authorization). Jika API Anda hanya memakai session + CSRF, Postman akan sulit dipakai; sediakan endpoint token khusus untuk non-produksi.

Postman: login dan simpan token

```
// Request: POST {{base_url}}/api/auth/login/   -> tab Tests
pm.test("Login sukses (200)", () => pm.response.to.have.status(200));

const json = pm.response.json();
pm.test("Ada token", () => pm.expect(json.token).to.be.a("string").and.not.empty);

pm.collectionVariables.set("token", json.token);
// Collection > Authorization: API Key, key=Authorization, value=Token {{token}}
// (untuk JWT: Bearer {{token}})
```

Postman: tambah anak

```
// Request: POST {{base_url}}/api/anak/
// Body (raw JSON), variabel dinamis agar NIK tidak bentrok antar-run:
{
  "nama": "Anak Uji {{$timestamp}}",
  "nik": "53{{$randomInt}}{{$randomInt}}0001",
  "jenis_kelamin": "L",
  "tanggal_lahir": "2022-05-10",
  "nama_ibu": "Ibu Uji"
}

// Tab Tests:
pm.test("Dibuat (201)", () => pm.response.to.have.status(201));
pm.test("Respons cepat (< 800 ms)", () => pm.expect(pm.response.responseTime).to.be.below(800));
pm.test("Skema benar", () => {
  const d = pm.response.json();
  pm.expect(d).to.include.keys("id", "nama", "nik", "tanggal_lahir", "puskesmas");
});
pm.test("Tidak membocorkan header debug", () => pm.expect(pm.response.headers.has("X-Debug")).to.be.false);
pm.collectionVariables.set("anak_id", pm.response.json().id);
```

Newman

```
# Newman: jalankan koleksi dari terminal/CI
npm i -g newman newman-reporter-htmlextra
newman run qa/postman/anak.postman_collection.json \
       -e qa/postman/staging.postman_environment.json \
       --reporters cli,htmlextra --reporter-htmlextra-export laporan-api.html
```

PERHATIAN. Simpan koleksi dan environment di Git (`qa/postman`) tetapi **hapus nilai rahasia** (token, password) sebelum commit. Pakai variabel dari CI untuk nilai rahasia.

3.5 Supertest: API testing dari sisi Node.js

Supertest dipakai bila ada layanan Node/Express (gateway, layanan laporan, mock EMIS) di samping Django. Ia memanggil aplikasi Node langsung tanpa membuka port. Untuk API Django sendiri, padanan terbaiknya adalah APIClient di bab 3.3.

Supertest

```
// tests/anak.api.test.js
import request from 'supertest';
import { describe, it, expect, beforeAll } from 'vitest';
import app from '../src/app.js';
```

```
let token;
beforeAll(async () => {
  const res = await request(app).post('/api/login')
    .send({ username: 'petugas_qa', password: 'Rahasia-QA-123' }).expect(200);
  token = res.body.token;
});

describe('POST /api/anak', () => {
  it('menyimpan anak baru', async () => {
    const res = await request(app).post('/api/anak')
      .set('Authorization', `Token ${token}`)
      .send({ nama: 'Joao Uji', nik: '5301010101200099', jenis_kelamin: 'L', tanggal_lahir: '2021-02-01' })
      .expect(201);
    expect(res.body.nama).toBe('Joao Uji');
  });

  it('menolak tanpa token', async () => {
    await request(app).post('/api/anak').send({}).expect(401);
  });
});
```

3.6 Kapan memakai yang mana

Alat	Kelebihan	Kekurangan	Dipakai untuk
DRF APIClient (pytest)	Cepat, ikut CI, akses DB langsung	Hanya untuk aplikasi Django	Regresi harian oleh programmer
Postman/Newman	Visual, mudah dibagi ke QA & klien, mendekati kondisi staging	Tidak bisa cek DB langsung	Eksplorasi, dokumentasi, smoke test staging
Supertest	Pas untuk layanan Node	Perlu kode JS	Layanan Node / BFF

3.7 Latihan bab 3

1. Lengkapi 6 skenario checklist untuk endpoint PUT `/api/anak/{id}/` (ubah data).
2. Buat koleksi Postman 'Alur Anak': login → tambah → ambil detail → ubah → hapus, dengan variabel `anak_id` yang mengalir.
3. Jalankan koleksi itu dengan Newman dan simpan laporan HTML.
4. Tulis test yang membuktikan 'admin kabupaten A' tidak bisa membaca laporan kabupaten B.

Bab 4 — Functional Testing (Manual + Selenium)

4.1 Konsep

Functional testing memeriksa bahwa **fitur memenuhi kebutuhan user**, bukan sekadar kode tidak error. Sumbernya adalah *requirement*/user story, mis. *"Sebagai guru, saya ingin menginput data EMIS siswa agar tersedia di laporan sekolah."* Pengujian ini sering dilakukan manual lebih dulu, lalu bagian yang stabil diotomatiskan.

4.2 Dari requirement ke test case

1. Tulis kebutuhan sebagai user story + **acceptance criteria** yang bisa diperiksa ya/tidak.
2. Turunkan jadi test case: kondisi awal, langkah, data, hasil yang diharapkan.
3. Sertakan jalur normal, jalur alternatif, dan jalur gagal.
4. Petakan di **traceability matrix** agar tidak ada requirement yang terlewat.

Contoh acceptance criteria: Guru input data EMIS

- Guru yang login melihat menu 'Data EMIS'; role lain tidak melihatnya.
- Form wajib: nama siswa, NISN, kelas, tanggal lahir, jenis kelamin. Kolom kosong menampilkan pesan jelas.
- Setelah Simpan, muncul pesan sukses dan data tampil di daftar siswa kelas tersebut.
- Guru hanya bisa melihat/mengubah siswa di sekolahnya sendiri.
- Tanggal lahir di masa depan ditolak.

4.3 Contoh test case manual

ID	Skenario	Langkah	Data	Hasil diharapkan	Status
FT-EMIS-01	Guru input siswa valid	1. Login guru 2. Menu Data EMIS 3. Klik Tambah 4. Isi form 5. Simpan	Nama: Joao Pereira; NISN: 0012345678; Kelas 3A	Pesan 'Data tersimpan'; Joao muncul di daftar kelas 3A	Lulus/ Gagal
FT-EMIS-02	Kolom wajib kosong	Seperti FT-01 tapi kosongkan Nama	Nama kosong	Pesan 'Nama wajib diisi'; data tidak tersimpan	
FT-EMIS-03	NISN duplikat	Input siswa dengan NISN yang sudah ada	NISN 0012345678	Ditolak dengan pesan duplikat	
FT-EMIS-04	Guru lain sekolah	Login guru sekolah B, buka siswa sekolah A (ubah ID di URL)	ID siswa sekolah A	Ditolak (403/404); data tidak tampil	
FT-EMIS-05	Role petugas puskesmas	Login petugas, cari menu Data EMIS	-	Menu tidak tampil; akses URL langsung ditolak	

4.4 Teknik merancang data uji

Teknik	Penjelasan	Contoh di form anak
Equivalence partitioning	Bagi input jadi kelompok yang diperlakukan sama; uji satu wakil tiap kelompok	Umur: negatif / 0-5 th / 6-18 th / >18 th
Boundary value	Uji tepat di batas	NIK 15, 16, 17 digit; umur 4 th 11 bln vs 5 th
Decision table	Kombinasi kondisi → aksi	Role x kepemilikan data → boleh/tidak
Error guessing	Tebak kesalahan umum user	Spasi di depan nama, tanda petik, huruf Portugis/Tetum (ã, ç, é)

TIPS PRO. Untuk Timor-Leste, uji juga **karakter khusus** (nama dengan tilde/aksen), format tanggal dd/mm/yyyy, dan teks dalam Bahasa Tetum/Portugis/Inggris jika antarmuka multi-bahasa. Pastikan database PostgreSQL berenkode UTF-8.

4.5 Selenium: mengotomatiskan test fungsional

Selenium menggerakkan browser sungguhan lewat WebDriver. Cocok bila tim sudah memakai Python dan butuh dukungan browser lama. Keunggulan di Django: fixture `live_server` dari pytest-django menjalankan server uji sungguhan dengan database uji, jadi data dibuat lewat factory dan **tidak perlu staging**.

Selenium + pytest + live_server

```
# pip install selenium
# anak/tests/test_emis_guru_selenium.py
import time
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from users.tests.factories import UserFactory


@pytest.fixture
def driver():
    opts = webdriver.ChromeOptions()
    opts.add_argument("--headless=new")
    d = webdriver.Chrome(options=opts)
    yield d
    d.quit()


def tunggu(d, locator, t=10):
    return WebDriverWait(d, t).until(EC.visibility_of_element_located(locator))


@pytest.mark.django_db(transaction=True)
def test_guru_input_emis(driver, live_server):
    UserFactory(username="guru_x", role="guru", puskesmas=None)     # password: Rahasia-123

    driver.get(f"{live_server.url}/accounts/login/")
    driver.find_element(By.NAME, "username").send_keys("guru_x")
    driver.find_element(By.NAME, "password").send_keys("Rahasia-123")
    driver.find_element(By.CSS_SELECTOR, "button[type=submit]").click()

    driver.find_element(By.LINK_TEXT, "Data EMIS").click()
    driver.find_element(By.ID, "btn-tambah").click()
    nama = f"Siswa Uji {int(time.time())}"
    driver.find_element(By.NAME, "nama").send_keys(nama)
    driver.find_element(By.NAME, "nisn").send_keys(str(int(time.time()))[-10:])
    driver.find_element(By.NAME, "kelas").send_keys("3A")
    driver.find_element(By.CSS_SELECTOR, "form button[type=submit]").click()

    pesan = tunggu(driver, (By.CSS_SELECTOR, ".alert-success")).text
```

```
    assert "tersimpan" in pesan.lower()
    assert nama in driver.page_source
```

> **TIPS PRO.** Aturan Selenium yang menghemat berjam-jam: pakai **explicit wait** (WebDriverWait), jangan `time.sleep()`; beri atribut `id`/`data-testid` stabil pada elemen penting; jangan bergantung pada XPath panjang.

4.6 Traceability matrix

Requirement	Deskripsi	Test case	Otomatis?	Status
REQ-01	Petugas mendaftarkan anak	FT-ANAK-01..06, E2E-01	Ya (pytest + Playwright)	Lulus
REQ-02	Guru input data EMIS	FT-EMIS-01..05	Sebagian (Selenium)	Berjalan
REQ-03	Laporan per puskesmas	FT-LAP-01..04, E2E-01	Ya (Playwright)	Berjalan
REQ-04	Ekspor laporan Excel/PDF	FT-LAP-05..06	Manual	Belum

4.7 Latihan bab 4

1. Tulis 10 test case (jalur normal, alternatif, gagal) untuk fitur 'ubah data anak'.
2. Otomatiskan 2 di antaranya dengan Selenium + `live_server`.
3. Buat traceability matrix untuk 5 requirement proyek Anda.

Bab 5 — End-to-End (E2E) Testing dengan Playwright

> **WAJIB / BAHAYA.** Inilah bab terpenting untuk UNICEF/pemerintah: mereka menilai sistem dari **alur nyata** yang dipakai petugas. E2E membuktikan alur itu jalan dari ujung ke ujung, setiap kali ada perubahan kode.

5.1 Konsep

E2E menggerakkan browser sungguhan seperti user asli: buka halaman, isi form, klik, lalu memeriksa hasilnya. Contoh alur kunci: **Login → Input data anak → Simpan → Laporan muncul**. Karena lambat dan mahal dirawat, E2E hanya untuk **5-10 alur kritis**, bukan setiap validasi kecil.

5.2 Cypress vs Playwright vs Selenium

Aspek	Playwright	Cypress	Selenium
Browser	Chromium, Firefox, WebKit (Safari)	Chromium, Firefox, WebKit (eksperimental)	Semua, termasuk lama
Bahasa	TypeScript/JS, Python, Java, .NET	JavaScript/TypeScript	Python, Java, C#, JS, dll.
Auto-wait	Ya, bawaan	Ya	Manual (WebDriverWait)
Kecepatan & paralel	Sangat baik, paralel bawaan	Baik (paralel berbayar di cloud)	Lebih lambat
Alat debug	Trace Viewer, codegen, UI mode	Time travel di runner	Terbatas
Tab/domain berganda	Didukung	Terbatas	Didukung
Rekomendasi	**Pilihan utama**	Bagus untuk tim front-end JS	Jika sudah ada aset Selenium

5.3 Instalasi dan data uji

Terminal

```
mkdir e2e && cd e2e
npm init -y
npm i -D @playwright/test
npx playwright install --with-deps chromium     # unduh browser (sekali)

# database & data QA khusus E2E (BUKAN data dev/produksi)
createdb anak_emis_e2e
python manage.py migrate --settings=config.settings.e2e
python manage.py seed_qa --settings=config.settings.e2e
```

settings E2E

```
# config/settings/e2e.py
from .base import *
DEBUG = False
DATABASES["default"]["NAME"] = "anak_emis_e2e"
ALLOWED_HOSTS = ["127.0.0.1", "localhost"]
```

5.4 Konfigurasi

playwright.config.ts

```
// e2e/playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
```

```
  testDir: './tests',
  timeout: 30_000,
  expect: { timeout: 7_000 },
  fullyParallel: true,
  forbidOnly: !!process.env.CI,              // cegah test.only ikut ter-commit
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 2 : undefined,
  reporter: [['list'], ['html', { open: 'never' }]],
  use: {
    baseURL: process.env.BASE_URL ?? 'http://127.0.0.1:8000',
    trace: 'on-first-retry',                 // jejak lengkap untuk debug di CI
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },
  projects: [
    { name: 'setup', testMatch: /.*\.setup\.ts/ },
    { name: 'desktop', dependencies: ['setup'],
      use: { ...devices['Desktop Chrome'], storageState: 'playwright/.auth/petugas.json' } },
    { name: 'mobile', dependencies: ['setup'],        // banyak petugas memakai HP
      use: { ...devices['Pixel 7'], storageState: 'playwright/.auth/petugas.json' } },
  ],
  webServer: process.env.BASE_URL ? undefined : {
    command: 'python ../manage.py runserver 8000 --noreload --settings=config.settings.e2e',
    url: 'http://127.0.0.1:8000/accounts/login/',
    reuseExistingServer: !process.env.CI,
    timeout: 60_000,
  },
});
```

5.5 Login sekali, pakai berkali-kali

Login lewat UI di setiap test itu lambat. Cukup login **sekali** di project *setup*, simpan sesinya (`storageState`), lalu semua test memakainya.

Setup autentikasi

```
// e2e/tests/auth.setup.ts
import { test as setup, expect } from '@playwright/test';

setup('login sebagai petugas puskesmas', async ({ page }) => {
  await page.goto('/accounts/login/');
  await page.getByLabel('Username').fill(process.env.E2E_USER ?? 'petugas_qa');
  await page.getByLabel('Password').fill(process.env.E2E_PASS ?? 'Rahasia-QA-123');
  await page.getByRole('button', { name: 'Masuk' }).click();
  await expect(page).toHaveURL(/dashboard/);
  await page.context().storageState({ path: 'playwright/.auth/petugas.json' });
});
```

> **PERHATIAN.** Tambahkan `playwright/.auth/` ke `.gitignore`. File itu berisi sesi login.

5.6 Page Object: halaman sebagai kelas

Pisahkan 'bagaimana menemukan elemen' dari 'apa yang diuji'. Jika tombol berganti nama, Anda cukup mengubah satu tempat.

Page Object

```
// e2e/pages/anak.page.ts
import { Page, expect } from '@playwright/test';

export class AnakPage {
  constructor(private page: Page) {}

  async buka() { await this.page.goto('/anak/'); }

  async tambah(d: { nama: string; nik: string; jk: 'L' | 'P'; lahir: string; ibu: string }) {
    await this.page.getByRole('link', { name: 'Tambah Anak' }).click();
    await this.page.getByLabel('Nama').fill(d.nama);
    await this.page.getByLabel('NIK').fill(d.nik);
    await this.page.getByLabel('Jenis Kelamin').selectOption(d.jk);
    await this.page.getByLabel('Tanggal Lahir').fill(d.lahir);       // format yyyy-mm-dd
```

```
    await this.page.getByLabel('Nama Ibu').fill(d.ibu);
    await this.page.getByRole('button', { name: 'Simpan' }).click();
  }

  async harusAdaDiDaftar(nama: string) {
    await this.buka();
    await expect(this.page.getByRole('row', { name: new RegExp(nama) })).toBeVisible();
  }
}

// e2e/pages/laporan.page.ts
import { Page, expect } from '@playwright/test';

export class LaporanPage {
  constructor(private page: Page) {}

  async jumlahBalita(): Promise<number> {
    await this.page.goto('/laporan/ringkasan/');
    const teks = await this.page.getByTestId('total-balita').innerText();   // <span data-testid="total-balita">
    return parseInt(teks.replace(/\D/g, ''), 10);
  }
}
```

5.7 Alur utama: Login → Input anak → Simpan → Laporan muncul

Test alur lengkap

```
// e2e/tests/alur-anak.spec.ts
import { test, expect } from '@playwright/test';
import { AnakPage } from '../pages/anak.page';
import { LaporanPage } from '../pages/laporan.page';

test.describe('Alur pendaftaran anak sampai laporan', () => {
  test('petugas mendaftarkan balita dan angka laporan bertambah', async ({ page }) => {
    const anak = new AnakPage(page);
    const laporan = new LaporanPage(page);

    // data unik tiap run supaya tidak bentrok dengan run sebelumnya
    const stamp = Date.now().toString();
    const nama = `Balita Uji ${stamp}`;
    const nik = `53010101${stamp.slice(-8)}`;                // 16 digit
    const lahir = new Date(Date.now() - 1000 * 60 * 60 * 24 * 365 * 2).toISOString().slice(0, 10); // ~2 th

    const sebelum = await laporan.jumlahBalita();            // 1. baca angka awal

    await page.goto('/dashboard/');                          // 2. sudah login lewat storageState
    await anak.tambah({ nama, nik, jk: 'P', lahir, ibu: 'Ibu Uji' });   // 3. isi form + simpan

    await expect(page.getByText('Data anak tersimpan')).toBeVisible();  // 4. umpan balik ke user
    await anak.harusAdaDiDaftar(nama);                                  // 5. muncul di daftar

    const sesudah = await laporan.jumlahBalita();            // 6. laporan ikut berubah
    expect(sesudah).toBe(sebelum + 1);
  });

  test('NIK tidak valid menampilkan pesan dan tidak menyimpan', async ({ page }) => {
    await page.goto('/anak/tambah/');
    await page.getByLabel('Nama').fill('Anak Salah NIK');
    await page.getByLabel('NIK').fill('123');
    await page.getByRole('button', { name: 'Simpan' }).click();
    await expect(page.getByText(/NIK.*16 digit/i)).toBeVisible();
  });
});

// e2e/tests/login.spec.ts   (test login: JANGAN pakai sesi tersimpan)
test.describe('Login', () => {
  test.use({ storageState: { cookies: [], origins: [] } });

  test('password salah menampilkan pesan error', async ({ page }) => {
    await page.goto('/accounts/login/');
    await page.getByLabel('Username').fill('petugas_qa');
    await page.getByLabel('Password').fill('salah');
    await page.getByRole('button', { name: 'Masuk' }).click();
    await expect(page.getByText(/tidak (valid|cocok)|salah/i)).toBeVisible();
    await expect(page).toHaveURL(/accounts\/login/);
```

```
  });

  test('halaman terlindungi mengarahkan ke login', async ({ page }) => {
    await page.goto('/anak/');
    await expect(page).toHaveURL(/accounts\/login/);
  });
});
```

5.8 Menjalankan dan men-debug

Tujuan	Perintah
Jalankan semua (headless)	npx playwright test
Satu file	npx playwright test tests/alur-anak.spec.ts
Lihat browser berjalan	npx playwright test --headed
UI mode (pilih & ulang test, time-travel)	npx playwright test --ui
Debug langkah demi langkah	npx playwright test --debug
Rekam klik jadi kode (awal yang bagus)	npx playwright codegen http://127.0.0.1:8000
Buka laporan HTML	npx playwright show-report
Buka trace dari CI	npx playwright show-trace trace.zip

5.9 Praktik pro E2E

- **Locator berbasis makna**: `getByRole`, `getByLabel`, `getByTestId`. Hindari CSS/XPath rapuh.
- **Jangan pakai sleep**. Playwright menunggu otomatis; gunakan `expect(...).toBeVisible()` sebagai sinkronisasi.
- **Test mandiri**: tiap test membuat datanya sendiri (nama unik) dan tidak bergantung urutan test lain.
- **Cukup alur kritis**. Validasi detail dipindah ke test API/unit yang lebih cepat.
- **Jangan skip test flaky**: pelajari trace, perbaiki akar masalah (biasanya data atau sinkronisasi).
- **Uji mobile**: banyak petugas puskesmas memakai HP dengan layar kecil dan koneksi lambat.
- **Simpan bukti**: trace, screenshot, dan video otomatis diunggah sebagai artefak CI. Bagus untuk laporan ke UNICEF.

5.10 Varian Python: pytest-playwright + live_server

Jika tim hanya menguasai Python, Playwright punya versi Python. Keuntungannya, `live_server` Django memakai database uji dan factory yang sama dengan test lain.

Python

```python
# pip install pytest-playwright && playwright install chromium
# pytest.ini tambahkan:  env = DJANGO_ALLOW_ASYNC_UNSAFE=true   (atau export di shell/CI)
import pytest
from playwright.sync_api import Page, expect
from users.tests.factories import UserFactory


@pytest.mark.django_db(transaction=True)
def test_alur_daftar_anak(page: Page, live_server):
    UserFactory(username="petugas_x")                          # password Rahasia-123

    page.goto(f"{live_server.url}/accounts/login/")
    page.get_by_label("Username").fill("petugas_x")
    page.get_by_label("Password").fill("Rahasia-123")
    page.get_by_role("button", name="Masuk").click()
    expect(page).to_have_url(f"{live_server.url}/dashboard/")

    page.get_by_role("link", name="Tambah Anak").click()
    page.get_by_label("Nama").fill("Anak Uji")
```

```
    page.get_by_label("NIK").fill("5301010101200777")
    page.get_by_label("Tanggal Lahir").fill("2022-05-10")
    page.get_by_role("button", name="Simpan").click()

    expect(page.get_by_text("Data anak tersimpan")).to_be_visible()
```

> **PERHATIAN.** Pytest-playwright memakai API sinkron di dalam event loop sehingga ORM Django bisa menolak dengan `SynchronousOnlyOperation`. Setel `DJANGO_ALLOW_ASYNC_UNSAFE=true` **hanya untuk lingkungan test**.

5.11 Latihan bab 5

1. Pasang Playwright dan rekam alur login dengan `codegen`, lalu rapikan jadi Page Object.
2. Tulis alur lengkap Login → Input anak → Simpan → Laporan di proyek Anda.
3. Tambah test: guru tidak bisa membuka halaman petugas (mengharapkan 403).
4. Sengaja rusak tombol Simpan; ambil trace dan temukan penyebabnya dengan `show-trace`.

Bab 6 — Performance Testing (k6, JMeter)

6.1 Konsep

Pertanyaannya sederhana: *"Apakah website tetap cepat dan tidak error saat semua puskesmas memakainya bersamaan?"* Jawabannya harus berupa angka, bukan perasaan.

6.2 Jenis uji beban

Jenis	Tujuan	Pola beban	Contoh
Smoke	Pastikan skrip & sistem hidup	1-5 user, 1 menit	Sebelum uji besar
Load	Beban normal & puncak yang diharapkan	Naik bertahap sampai target, tahan 10-30 menit	1000 user (semua puskesmas)
Stress	Cari titik patah	Naikkan terus sampai gagal	2000-3000 user
Spike	Lonjakan mendadak	0 → tinggi dalam detik	Jam 08.00 semua login
Soak	Kebocoran memori/koneksi	Beban sedang 4-12 jam	Semalam penuh

6.3 Dari '1000 orang' menjadi angka teknis

1000 pengguna terdaftar bukan **1000 permintaan per detik**. Gunakan rumus:

RPS ≈ pengguna aktif ÷ (waktu berpikir + waktu respons). Contoh: 1000 pengguna aktif, waktu berpikir rata-rata 8 detik, respons 0,5 detik → ≈ 117 permintaan/detik. Tetap uji juga skenario buruk (3x lipat) sebagai margin.

6.4 Menulis skrip k6

k6 memakai JavaScript, ringan, dan mudah dimasukkan ke CI. Skrip di bawah mensimulasikan petugas yang login satu kali, lalu bergantian melihat daftar, membuka laporan, dan sesekali menambah anak.

k6 script

```
// qa/k6/load.js
import http from 'k6/http';
import { check, sleep, group } from 'k6';

const BASE = __ENV.BASE_URL;                    // WAJIB staging, bukan produksi

export const options = {
  stages: [
    { duration: '2m', target: 200 },            // pemanasan
    { duration: '5m', target: 1000 },           // naik ke 1000 pengguna virtual
    { duration: '10m', target: 1000 },          // tahan di puncak
    { duration: '2m', target: 0 },              // turun
  ],
  thresholds: {                                 // gagal otomatis jika dilanggar
    http_req_failed:   ['rate<0.01'],           // error kurang dari 1%
    http_req_duration: ['p(95)<800', 'p(99)<2000'],
    checks:            ['rate>0.99'],
  },
};

let token = null;                               // satu token per VU

function login() {
  const n = (__VU % 50) + 1;                    // pool 50 akun uji: load_01 .. load_50
  const user = `load_${String(n).padStart(2, '0')}`;
  const r = http.post(`${BASE}/api/auth/login/`,
    JSON.stringify({ username: user, password: __ENV.LOAD_PASS }),
    { headers: { 'Content-Type': 'application/json' } });
  check(r, { 'login 200': (x) => x.status === 200 });
```

```
  return r.json('token');
}

export default function () {
  if (!token) token = login();
  const h = { headers: { Authorization: `Token ${token}`, 'Content-Type': 'application/json' } };

  group('lihat daftar anak', () => {
    const r = http.get(`${BASE}/api/anak/?page=1`, h);
    check(r, { 'daftar 200': (x) => x.status === 200 });
  });
  sleep(Math.random() * 3 + 2);                  // waktu berpikir 2-5 detik

  group('buka laporan', () => {
    const r = http.get(`${BASE}/api/laporan/ringkasan/`, h);
    check(r, { 'laporan 200': (x) => x.status === 200 });
  });
  sleep(Math.random() * 3 + 2);

  if (Math.random() < 0.2) {                     // 20% iterasi menambah data
    group('tambah anak', () => {
      const nik = `53${String(__VU).padStart(4, '0')}${String(__ITER).padStart(6, '0')}0001`.slice(0, 16);
      const r = http.post(`${BASE}/api/anak/`, JSON.stringify({
        nama: `Anak Beban ${__VU}-${__ITER}`, nik, jenis_kelamin: 'L',
        tanggal_lahir: '2022-05-10', nama_ibu: 'Ibu Uji' }), h);
      check(r, { 'tambah 201': (x) => x.status === 201 });
    });
    sleep(2);
  }
}
```

Menjalankan

```
# Smoke dulu (1 user), baru beban penuh
k6 run -e BASE_URL=https://staging.example.org -e LOAD_PASS=xxxx --vus 1 --duration 30s qa/k6/load.js
k6 run -e BASE_URL=https://staging.example.org -e LOAD_PASS=xxxx qa/k6/load.js

# Simpan hasil untuk laporan
k6 run --out json=hasil.json qa/k6/load.js
```

PERHATIAN. Siapkan akun beban dan data awal dengan management command khusus (mis. `python manage.py seed_load --users 50 --anak 20000`). Database kosong memberi hasil yang terlalu bagus; uji dengan volume data yang mendekati produksi.

PERHATIAN. Aturan pembatasan login (throttling) akan menolak ribuan login dari satu IP uji. Longgarkan hanya di staging saat uji beban, dan **jangan** longgarkan di produksi.

6.5 Membaca hasil

Metrik	Arti	Target contoh
http_req_duration p(95)	95% permintaan lebih cepat dari angka ini	< 800 ms
p(99)	Kasus terburuk yang masih umum	< 2 detik
http_req_failed	Persentase gagal (5xx, timeout)	< 1%
http_reqs (RPS)	Throughput	Sesuai perhitungan 6.3
vus / iterations	Pengguna virtual / siklus selesai	Naik linear, tidak mendatar
CPU, RAM, koneksi DB (server)	Kapasitas sumber daya	CPU < 75%, koneksi DB tidak habis

Kurva sehat: waktu respons datar saat pengguna naik. Tanda masalah: waktu respons **melonjak tajam** di titik tertentu (saturasi), error muncul bergelombang (pool koneksi habis), atau memori naik terus (kebocoran).

6.6 Titik penyempitan umum di Django + PostgreSQL

Gejala	Penyebab lazim	Perbaikan
Daftar & laporan lambat	N+1 query, tanpa index	`select_related`/`prefetch_related`, `db_index=True`, cek `EXPLAIN ANALYZE`
Error 'too many connections'	Setiap worker membuka koneksi baru	PgBouncer, `CONN_MAX_AGE`, naikkan `max_connections` secukupnya
Satu request memblokir banyak user	Worker gunicorn terlalu sedikit/ekspor berat	Gunicorn workers ≈ 2xCPU+1, ekspor ke Celery
Laporan agregat berat	Hitung ulang setiap dibuka	Cache (Redis), tabel ringkasan, materialized view
Daftar besar lambat	Tanpa paginasi	Paginasi DRF, filter wajib, batasi ukuran halaman
Static file memperlambat	Django melayani file statis	Nginx/WhiteNoise/CDN
Query lambat tak terlihat	Tak ada pemantauan	`pg_stat_statements`, django-silk (staging), log slow query

6.7 JMeter: alternatif berbasis GUI

JMeter cocok bila tim lebih nyaman dengan antarmuka grafis atau butuh protokol selain HTTP. Alur: buat *Thread Group* (jumlah user, ramp-up) → *HTTP Request* → *Assertion* → *Listener* untuk grafik. Rancang di GUI, jalankan di mode CLI agar hemat sumber daya:

JMeter CLI

```
jmeter -n -t qa/jmeter/anak.jmx -l hasil.jtl -e -o laporan-jmeter/
```

6.8 Aturan main uji beban

- Hanya di **staging** yang spesifikasinya mirip produksi. Beritahu tim hosting/DevOps sebelum mulai.
- Jangan menembak produksi tanpa jadwal dan persetujuan tertulis; bisa dianggap serangan DoS oleh penyedia hosting.
- Gunakan data palsu. Hapus data beban setelah selesai.
- Jalankan generator beban dari mesin yang cukup kuat (CPU/jaringan), kalau tidak yang jenuh justru alat ujinya.
- Ulangi setelah setiap optimasi dan catat perbandingannya (sebelum → sesudah).

6.9 Latihan bab 6

1. Jalankan smoke test k6 dengan 1 VU, lalu 50 VU, lalu 200 VU. Catat p95 di setiap tahap.
2. Temukan satu endpoint paling lambat dan perbaiki (index atau select_related). Ukur ulang.
3. Hitung target RPS sistem Anda dengan rumus 6.3 untuk jumlah puskesmas sebenarnya.

Bab 7 — Security Testing (OWASP ZAP) — WAJIB untuk Data Anak

WAJIB / BAHAYA. Data anak termasuk data sensitif. Kebocoran bisa membahayakan anak dan keluarga serta merusak kepercayaan UNICEF dan pemerintah. Keamanan bukan fase akhir; ia diuji sejak awal dan setiap rilis.

7.1 Aturan keterlibatan (rules of engagement)

- Hanya uji sistem yang **Anda miliki izin tertulisnya** untuk diuji (dari pemilik sistem/klien).
- Hanya di **staging** dengan data palsu. Scan aktif ke produksi dapat merusak atau memicu data sampah.
- Beritahu penyedia hosting jika perlu. Catat jadwal, IP penguji, dan ruang lingkup.
- Temuan kerentanan bersifat rahasia: kirim hanya ke orang yang berwenang.

7.2 OWASP Top 10 dan padanannya di Django

Risiko OWASP	Contoh di sistem data anak	Pertahanan Django/PostgreSQL
A01 Broken Access Control	Petugas A membuka data anak puskesmas B dengan mengubah ID di URL (IDOR)	Filter queryset per puskesmas, permission class, test IDOR (bab 3)
A02 Cryptographic Failures	NIK/identitas tersimpan polos, HTTP tanpa TLS	HTTPS wajib, HSTS, enkripsi kolom sensitif, backup terenkripsi
A03 Injection	SQL injection di pencarian, XSS di kolom nama	ORM (hindari `raw()`/`extra()`), autoescape template, jangan `|safe` data user
A04 Insecure Design	Tidak ada batas percobaan login, tidak ada audit	Throttling, audit log, threat modeling
A05 Security Misconfiguration	DEBUG=True di server, SECRET_KEY di repo	`manage.py check --deploy`, env var, ALLOWED_HOSTS ketat
A06 Vulnerable Components	Django/paket lama berkerentanan	`pip-audit`, Dependabot, pembaruan rutin
A07 Auth Failures	Password lemah, brute force	Password validators, django-axes / DRF throttle, 2FA untuk admin
A08 Data Integrity	Impor Excel berisi payload jahat	Validasi ketat berkas & isi, batasi ukuran/tipe
A09 Logging Failures	Tak ada jejak siapa melihat data anak; atau NIK tertulis di log	Audit log (django-auditlog/simple-history), jangan log data pribadi
A10 SSRF	Fitur impor dari URL mengakses jaringan internal	Daftar putih domain, batasi egress

7.3 Pengecekan otomatis yang murah dan efektif

Jalankan di CI setiap commit

```
# 1) Konfigurasi Django siap produksi?
python manage.py check --deploy --settings=config.settings.prod

# 2) Analisis kode statis (SQL injection, eval, password hard-coded, dll.)
bandit -r . -x ./tests,./e2e -ll

# 3) Dependensi berkerentanan
pip-audit -r requirements.txt

# 4) Cari rahasia yang tak sengaja ter-commit
gitleaks detect --source .
```

settings produksi

```
# config/settings/prod.py  - minimal untuk data sensitif
import os
DEBUG = False
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]                 # JANGAN di repo
ALLOWED_HOSTS = os.environ["ALLOWED_HOSTS"].split(",")

SECURE_SSL_REDIRECT = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_CONTENT_TYPE_NOSNIFF = True
X_FRAME_OPTIONS = "DENY"
SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")   # jika di belakang Nginx/LB

AUTH_PASSWORD_VALIDATORS = [
    {"NAME": "django.contrib.auth.password_validation.MinimumLengthValidator", "OPTIONS": {"min_length": 12}},
    {"NAME": "django.contrib.auth.password_validation.CommonPasswordValidator"},
    {"NAME": "django.contrib.auth.password_validation.NumericPasswordValidator"},
]

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": ["rest_framework.authentication.TokenAuthentication"],
    "DEFAULT_PERMISSION_CLASSES": ["rest_framework.permissions.IsAuthenticated"],   # tertutup secara default
    "DEFAULT_THROTTLE_CLASSES": ["rest_framework.throttling.AnonRateThrottle",
                                 "rest_framework.throttling.UserRateThrottle"],
    "DEFAULT_THROTTLE_RATES": {"anon": "20/min", "user": "300/min"},
}
```

7.4 Test keamanan sebagai kode (pytest)

Test keamanan

```
# anak/tests/test_security.py
import pytest
from users.tests.factories import AnakFactory

pytestmark = pytest.mark.django_db


def test_xss_nama_ditampilkan_ter_escape(client, petugas):
    AnakFactory(nama="<script>alert(1)</script>", puskesmas=petugas.puskesmas)
    client.force_login(petugas)

    html = client.get("/anak/").content.decode()

    assert "<script>alert(1)</script>" not in html          # tidak boleh mentah
    assert "&lt;script&gt;" in html


def test_sql_injection_pada_pencarian_tidak_merusak(petugas_client, petugas):
    AnakFactory.create_batch(3, puskesmas=petugas.puskesmas)

    resp = petugas_client.get("/api/anak/", {"search": "x' OR '1'='1"})

    assert resp.status_code == 200
    assert resp.json()["count"] == 0                        # diperlakukan sebagai teks biasa


def test_login_dibatasi_setelah_banyak_percobaan(api_client, django_user_model, settings):
    settings.REST_FRAMEWORK = {**settings.REST_FRAMEWORK,
                               "DEFAULT_THROTTLE_RATES": {"anon": "5/min", "user": "300/min"}}
    for _ in range(5):
        api_client.post("/api/auth/login/", {"username": "x", "password": "salah"}, format="json")

    resp = api_client.post("/api/auth/login/", {"username": "x", "password": "salah"}, format="json")

    assert resp.status_code == 429                          # Too Many Requests


def test_respons_tidak_membocorkan_field_internal(petugas_client, petugas):
    anak = AnakFactory(puskesmas=petugas.puskesmas)
    data = petugas_client.get(f"/api/anak/{anak.id}/").json()
```

```
    assert not {"password", "created_by_ip", "catatan_internal"} & set(data)


def test_endpoint_tertutup_tanpa_token(api_client):
    for url in ["/api/anak/", "/api/laporan/ringkasan/", "/api/emis/siswa/"]:
        assert api_client.get(url).status_code in (401, 403), url
```

> **TIPS PRO.** Test `test_endpoint_tertutup_tanpa_token` sebaiknya memuat **seluruh** URL API yang ada. Idealnya otomatis dari daftar URL router, sehingga endpoint baru yang lupa dilindungi langsung terdeteksi.

7.5 OWASP ZAP: pemindai kerentanan

ZAP (Zed Attack Proxy) berfungsi sebagai 'penyerang otomatis'. Ia menjelajah situs lalu menguji pola serangan umum. Mode yang dipakai:

Mode	Fungsi	Aman untuk	Kapan
Baseline scan	Pasif: membaca respons, memeriksa header, cookie, info bocor	Staging & non-produksi	Setiap build / mingguan
API scan	Membaca spesifikasi OpenAPI lalu menguji tiap endpoint	Staging	Sebelum rilis
Full (active) scan	Aktif: mencoba injeksi, XSS, dll.	**Hanya staging** berdata palsu	Sebelum rilis besar & audit
Mode manual (GUI proxy)	Anda klik-klik sambil ZAP merekam & menguji	Staging	Untuk alur login/form kompleks

ZAP via Docker

```
mkdir -p qa/zap && chmod 777 qa/zap          # kontainer ZAP perlu menulis laporan

# 1) Baseline (cepat, pasif)
docker run --rm -v "$(pwd)/qa/zap:/zap/wrk:rw" -t ghcr.io/zaproxy/zaproxy:stable \
  zap-baseline.py -t https://staging.example.org -r zap-baseline.html -J zap-baseline.json

# 2) API scan memakai skema OpenAPI (mis. dari drf-spectacular: /api/schema/)
docker run --rm -v "$(pwd)/qa/zap:/zap/wrk:rw" -t ghcr.io/zaproxy/zaproxy:stable \
  zap-api-scan.py -t https://staging.example.org/api/schema/ -f openapi -r zap-api.html

# 3) Full scan (aktif) - HANYA staging dengan izin
docker run --rm -v "$(pwd)/qa/zap:/zap/wrk:rw" -t ghcr.io/zaproxy/zaproxy:stable \
  zap-full-scan.py -t https://staging.example.org -r zap-full.html
```

Scan dengan login. Baseline tanpa login hanya melihat halaman publik. Untuk menguji area yang ada data anak, pakai ZAP Desktop: jalankan alur login lewat browser yang diproxy-kan ZAP, buat *Context* + *User* + *Authentication*, lalu aktifkan *Forced User Mode*. Dokumentasikan akun yang dipakai (akun QA, bukan akun asli).

7.6 Triase temuan

Tingkat ZAP	Tindakan	Tenggat
High	Perbaiki sebelum rilis, tulis test regresi	Segera / blokir rilis
Medium	Perbaiki di sprint berjalan atau jelaskan mitigasi	≤ 2 minggu
Low	Evaluasi; sering header kecil yang mudah diperbaiki	Sprint berikutnya
Informational	Baca; abaikan jika tidak relevan (catat alasan)	-

Hati-hati dengan **false positive**: verifikasi manual sebelum melaporkan. Sebaliknya, ZAP tidak menemukan **logika bisnis** yang salah (mis. IDOR). Itu sebabnya test otorisasi di bab 3 & 7.4 tetap wajib.

7.7 Keamanan khusus data anak

- **Minimasi data**: simpan hanya yang benar-benar diperlukan program.
- **Enkripsi** saat transit (TLS) dan saat disimpan (disk/backup terenkripsi; pertimbangkan enkripsi kolom untuk NIK).
- **Akses berdasarkan peran & wilayah** (prinsip hak akses minimum); tinjau akun secara berkala dan matikan akun yang tak dipakai.
- **Audit log**: siapa membuka/mengubah/mengekspor data anak, kapan.
- **Pengguna PostgreSQL** aplikasi hanya punya hak yang perlu (bukan superuser); koneksi memakai SSL; port tidak terbuka ke publik.
- **Backup** terenkripsi, diuji pulih (restore) secara berkala, disimpan terpisah.
- **Data uji** tidak boleh berasal dari data nyata; jika terpaksa, anonimkan dahulu.
- **Rencana insiden**: siapa dihubungi, apa yang dilakukan, kapan memberi tahu UNICEF/pemerintah.
- Ikuti kebijakan perlindungan data UNICEF dan peraturan setempat; konfirmasikan persyaratannya dengan klien.

7.8 Checklist keamanan sebelum go-live

	Item	Status
[ ]	DEBUG=False, SECRET_KEY dari env, ALLOWED_HOSTS benar	
[ ]	HTTPS aktif, HSTS, cookie secure, `check --deploy` bersih	
[ ]	Semua endpoint butuh autentikasi kecuali yang sengaja publik (daftar tertulis)	
[ ]	Test IDOR/otorisasi lulus untuk setiap role	
[ ]	Throttling login & API aktif, password policy kuat	
[ ]	ZAP baseline + API scan tanpa High; Medium sudah dijelaskan	
[ ]	pip-audit & bandit tanpa temuan serius	
[ ]	Audit log berjalan; log tidak memuat data pribadi	
[ ]	Backup terenkripsi & uji restore berhasil	
[ ]	Akun admin memakai 2FA; akun QA/default dihapus	

7.9 Latihan bab 7

1. Jalankan `check --deploy`, `bandit`, dan `pip-audit`; perbaiki temuan.
2. Jalankan ZAP baseline ke staging, simpan laporan, dan tulis 3 temuan beserta rencana perbaikan.
3. Tulis test IDOR untuk semua endpoint yang menerima ID.
4. Buat dokumen 1 halaman 'Apa yang kami lakukan jika data anak bocor'.

Bab 8 — User Acceptance Testing (UAT)

8.1 Konsep

UAT adalah tes terakhir, dilakukan oleh **orang UNICEF / Pemerintah Timor-Leste** (dan pengguna sebenarnya: petugas puskesmas, guru). Mereka mencoba sistem dan menyatakan *"OK, lulus"*. Tugas Anda sebagai programmer adalah **menyiapkan UAT sehingga mereka bisa menilai dengan mudah**, bukan ikut menilai.

8.2 Syarat masuk UAT (entry criteria)

- Semua test otomatis (unit, API, E2E) hijau di CI pada versi yang sama.
- Tidak ada bug **Critical/High** terbuka dari pengujian internal; ZAP tanpa temuan High.
- Lingkungan UAT terpisah, stabil, berisi **data palsu yang realistis**.
- Akun UAT untuk tiap peran sudah dibuat; panduan singkat dan skrip uji sudah dikirim.
- Catatan rilis (apa yang baru, apa yang belum ada) sudah tertulis.

8.3 Rencana UAT

Elemen	Isi
Tujuan	Memastikan sistem memenuhi kebutuhan program dan layak digunakan di lapangan
Peserta	Perwakilan UNICEF/pemerintah, 2-3 petugas puskesmas, 2-3 guru, admin kabupaten
Durasi	1-2 minggu; sesi pendampingan 2-3 jam per kelompok
Lingkungan	uat.example.org (data palsu); akun per peran
Bahasa	Skrip & antarmuka sesuai bahasa pengguna (Tetum/Portugis/Inggris)
Pelaporan masalah	Formulir bug (Lampiran B) atau spreadsheet bersama; PIC pihak programmer menjawab maksimal 1 hari kerja
Penutup	Rapat tinjau, daftar temuan, keputusan Go/No-Go, tanda tangan

8.4 Skrip skenario UAT (bahasa user, bukan bahasa teknis)

ID	Peran	Skenario	Langkah singkat	Hasil yang diharapkan	Lulus?
UAT-01	Petugas puskesmas	Mendaftarkan balita baru	Masuk → Tambah Anak → isi data → Simpan	Muncul pesan sukses; anak ada di daftar	[ ] Ya [ ] Tidak
UAT-02	Petugas puskesmas	Mencari anak berdasar nama	Kolom cari → ketik nama	Hasil sesuai; hanya anak puskesmas sendiri	[ ] Ya [ ] Tidak
UAT-03	Guru	Input data EMIS siswa	Menu Data EMIS → Tambah → Simpan	Siswa tampil di daftar kelas	[ ] Ya [ ] Tidak
UAT-04	Admin kabupaten	Melihat laporan bulanan	Menu Laporan → pilih bulan	Angka sesuai data yang dimasukkan	[ ] Ya [ ] Tidak
UAT-05	Admin kabupaten	Ekspor laporan ke Excel	Laporan → Ekspor	File terunduh, kolom & angka benar	[ ] Ya [ ] Tidak
UAT-06	Semua	Dipakai di HP dengan koneksi lambat	Ulangi UAT-01 di HP	Masih bisa dipakai tanpa error	[ ] Ya [ ] Tidak
UAT-07	Semua	Lupa/ganti password	Ikuti menu lupa password	Berhasil & aman	[ ] Ya [ ] Tidak

8.5 Definisi tingkat keparahan temuan

Tingkat	Definisi	Contoh	Aturan UAT
Critical	Sistem tak bisa dipakai / data bocor / data salah masuk laporan	Data anak tampil ke puskesmas lain	Go-live DITUNDA
High	Fitur utama gagal tanpa jalan keluar	Tidak bisa menyimpan data anak	Harus diperbaiki sebelum Go
Medium	Fitur gagal tetapi ada cara lain	Ekspor Excel gagal, PDF bisa	Disepakati: perbaiki / jadwalkan
Low	Kosmetik / kenyamanan	Typo, penjajaran tombol	Boleh setelah Go

8.6 Kriteria lulus (exit criteria)

- 100% skenario UAT inti dijalankan.
- Tidak ada temuan Critical/High terbuka. Medium memiliki rencana dan tanggal.
- Perwakilan klien menandatangani formulir penerimaan.
- Panduan pengguna dan pelatihan singkat sudah diberikan.

8.7 Formulir penerimaan (sign-off)

Template

```
FORMULIR PENERIMAAN UAT
------------------------------------------------------------
Sistem        : Website Data Anak & EMIS
Versi / build : _______________    Tanggal UAT : _______________
Lingkungan    : _______________

Ringkasan hasil
  Skenario dijalankan : ____ dari ____       Lulus : ____   Gagal : ____
  Temuan terbuka      : Critical ___  High ___  Medium ___  Low ___

Keputusan (pilih satu)
  [ ] DITERIMA                        - sistem siap go-live
  [ ] DITERIMA DENGAN CATATAN         - perbaikan disepakati (lihat daftar)
  [ ] DITOLAK                         - uji ulang setelah perbaikan

Catatan / syarat : _____________________________________________

Pihak penerima (UNICEF / Pemerintah)      Pihak pengembang
Nama    : _____________________           Nama    : _____________________
Jabatan : _____________________           Jabatan : _____________________
TTD     : _____________________           TTD     : _____________________
Tanggal : _____________________           Tanggal : _____________________
```

8.8 Tips mengelola UAT

- Dampingi, jangan mengarahkan: biarkan user mencoba sendiri; catat tempat mereka bingung (itu temuan kegunaan).
- Catat **semua** masukan, lalu klasifikasikan dengan klien; jangan debat saat sesi.
- Beri umpan balik cepat: tiap temuan punya status (Baru, Dikerjakan, Siap uji ulang, Selesai).
- Uji ulang hanya yang diperbaiki + regresi otomatis (E2E dan API) agar cepat.
- Simpan bukti (tangkapan layar, laporan Playwright) untuk arsip audit.

Bab 9 — Menyatukan Semuanya: CI/CD, Strategi, Roadmap

9.1 Pipeline CI (GitHub Actions)

Tujuan: setiap push/PR menjalankan pengujian otomatis, dan kode yang gagal tidak bisa digabung.

tests.yml

```
# .github/workflows/tests.yml
name: tests
on: [push, pull_request]

jobs:
  backend:                      # unit + integration + keamanan statis
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: anak_test
          POSTGRES_PASSWORD: ganti-ini
          POSTGRES_DB: anak_emis
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U anak_test" --health-interval 5s
          --health-timeout 5s --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: bandit -r . -x ./tests,./e2e -ll
      - run: pip-audit -r requirements.txt
      - run: pytest -n auto --cov=. --cov-fail-under=70
        env: { TEST_DB_HOST: 127.0.0.1, TEST_DB_USER: anak_test, TEST_DB_PASSWORD: ganti-ini }

  e2e:                          # Playwright
    runs-on: ubuntu-latest
    needs: backend
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_USER: anak_test, POSTGRES_PASSWORD: ganti-ini, POSTGRES_DB: anak_emis_e2e }
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U anak_test" --health-interval 5s
          --health-timeout 5s --health-retries 10
    env:
      CI: "true"
      DJANGO_SETTINGS_MODULE: config.settings.e2e
      TEST_DB_HOST: 127.0.0.1
      TEST_DB_USER: anak_test
      TEST_DB_PASSWORD: ganti-ini
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: pip install -r requirements.txt
      - run: python manage.py migrate && python manage.py seed_qa
      - run: npm ci && npx playwright install --with-deps chromium
        working-directory: e2e
      - run: npx playwright test
        working-directory: e2e
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: playwright-report, path: e2e/playwright-report }
```

Jangan lupa mengaktifkan **branch protection** di GitHub: wajibkan job *backend* dan *e2e* hijau sebelum merge.

9.2 Kapan setiap jenis tes dijalankan

Tes	Pemicu	Durasi target	Gerbang (gate)
Unit + API (pytest)	Setiap push / PR	< 5 menit	Wajib hijau untuk merge
Bandit, pip-audit	Setiap push / PR	< 2 menit	Tidak ada temuan serius
E2E Playwright	Setiap PR ke main, nightly	< 15 menit	Wajib hijau sebelum rilis
Postman/Newman (staging)	Setelah deploy ke staging	< 5 menit	Smoke test lulus
k6 beban	Sebelum rilis besar, bulanan	30-60 menit	Threshold terpenuhi
ZAP baseline / API	Mingguan + sebelum rilis	10-30 menit	Tidak ada High
ZAP full + manual	Sebelum go-live & audit	Beberapa jam	Tidak ada High/Critical
UAT	Sebelum go-live	1-2 minggu	Sign-off klien

9.3 Definition of Done (DoD) untuk setiap fitur

- Kode direview rekan.
- Unit test untuk logika baru; test API untuk 6 skenario endpoint.
- Jika alur kritis berubah, E2E diperbarui.
- Uji otorisasi dan IDOR untuk data baru.
- Tidak ada query N+1 pada daftar/laporan baru.
- Migrasi database diuji naik dan (bila perlu) turun.
- Dokumentasi/catatan rilis diperbarui.

9.4 Checklist rilis

	Item
[ ]	CI hijau di commit yang akan dirilis (pytest, E2E)
[ ]	Migrasi diuji pada salinan data (palsu) berukuran besar
[ ]	Backup database dibuat dan diuji pulih
[ ]	Smoke test Postman/Newman di staging lulus
[ ]	Hasil uji beban terakhir memenuhi threshold
[ ]	ZAP tanpa High; `check --deploy` bersih
[ ]	UAT sign-off diterima untuk versi ini
[ ]	Rencana rollback tertulis dan pernah dicoba
[ ]	Pemantauan & alarm (error rate, latensi) aktif
[ ]	Seluruh pihak terkait diberi tahu jadwal rilis

9.5 Metrik kualitas yang layak dilaporkan

Metrik	Target awal	Catatan
Coverage backend (baris)	≥ 70% (logika inti ≥ 85%)	Coverage bukan tujuan; jangan menulis test kosong
Pass rate CI di main	≥ 95%	Merah di main = diperbaiki hari itu juga
Flaky test	0 dibiarkan	Karantina sementara boleh ≤ 1 minggu dengan tiket
Bug lolos ke produksi / rilis	Turun tiap kuartal	Tiap bug produksi = 1 test regresi
Waktu respons p95	< 800 ms	Dari k6 & pemantauan produksi

9.6 Roadmap belajar 8 minggu untuk programmer lokal

Minggu	Fokus	Hasil yang dikumpulkan
1	Siapkan lingkungan (Bab 1) + dasar pytest	Proyek latihan, 10 test sederhana hijau
2	Unit test mendalam (Bab 2): parametrize, mock, boundary	Coverage logika inti ≥ 80%
3	API test DRF (Bab 3.1-3.3): 6 skenario per endpoint	Test lengkap untuk 3 endpoint
4	Postman + Newman (Bab 3.4)	Koleksi 'Alur Anak' + laporan HTML
5	Playwright dasar (Bab 5.1-5.6)	Login + 1 page object berjalan
6	Playwright alur lengkap + CI (Bab 5.7, 9.1)	Alur Login-Input-Simpan-Laporan hijau di CI
7	Keamanan (Bab 7) + beban (Bab 6)	Laporan ZAP baseline + hasil k6 smoke & load
8	Fungsional & UAT (Bab 4, 8) + portofolio	Paket UAT lengkap; repo contoh untuk portofolio

TIPS PRO. Tips karier: repositori publik berisi proyek Django kecil dengan test pytest, alur Playwright, dan laporan k6/ZAP (tanpa data nyata) adalah portofolio yang sangat kuat untuk melamar kerja di organisasi pembangunan dan perusahaan teknologi.

9.7 Template dokumen Strategi Pengujian (1-2 halaman)

Kerangka

```
STRATEGI PENGUJIAN - Website Data Anak & EMIS
1. Ruang lingkup   : modul, peran, lingkungan (dev / staging / UAT / produksi)
2. Risiko utama    : kebocoran data anak, kesalahan laporan, ketidaktersediaan saat jam puncak
3. Jenis & alat    : Unit (pytest) | API (DRF, Postman) | E2E (Playwright) | Beban (k6)
                     Keamanan (ZAP, bandit, pip-audit) | UAT (klien)
4. Data uji        : data palsu via factory & seed_qa; dilarang memakai data anak nyata
5. Kriteria masuk/keluar tiap tahap (lihat Bab 8, 9.2)
6. Peran           : programmer (unit/API/E2E), QA (fungsional), DevOps (beban), klien (UAT)
7. Pelaporan bug   : alat, tingkat keparahan, SLA respons
8. Jadwal          : tanggal uji beban, scan keamanan, UAT, go-live
9. Metrik          : coverage, pass rate, p95, jumlah temuan terbuka
```

Lampiran

A. Template test case

Field	Isi
ID	FT-<modul>-<nomor>
Judul	Kalimat singkat apa yang diuji
Requirement	REQ-xx
Prioritas	Tinggi / Sedang / Rendah
Prasyarat	Akun, data awal, kondisi sistem
Langkah	1. ... 2. ... 3. ...
Data uji	Nilai input yang dipakai
Hasil diharapkan	Apa yang harus terjadi (UI, data, pesan)
Hasil aktual	Diisi saat uji
Status	Lulus / Gagal / Terblokir
Bukti	Tangkapan layar / tautan trace
Penguji & tanggal	Nama, tanggal, versi build

B. Template laporan bug

Template bug

```
Judul        : [Modul] Ringkasan masalah dalam satu kalimat
ID / Tanggal : BUG-0123 / 2026-10-07          Dilaporkan oleh : _____________
Lingkungan   : staging | UAT | produksi       Versi/Build     : _____________
Peran akun   : petugas_puskesmas | guru | admin_kabupaten
Browser/Perangkat : Chrome 130 / Android 14 (HP)
Tingkat      : Critical | High | Medium | Low

Langkah mereproduksi
  1. Masuk sebagai petugas_qa
  2. Buka Anak > Tambah Anak
  3. Isi NIK 16 digit yang sudah ada, klik Simpan
Hasil yang diharapkan : Pesan 'NIK sudah terdaftar', data tidak tersimpan
Hasil aktual          : Halaman error 500
Bukti                 : tangkapan layar, log server, trace Playwright
Catatan               : Terjadi sejak build 2026-10-05. Tidak terjadi untuk NIK baru.
```

C. Contekan perintah

Alat	Perintah penting
pytest	pytest -x -k nama | pytest -n auto | pytest --create-db | pytest --cov=anak --cov-report=term-missing
Django	python manage.py check --deploy | migrate | makemigrations --check --dry-run
Playwright	npx playwright test | --ui | --debug | codegen URL | show-report | show-trace trace.zip
Newman	newman run koleksi.json -e env.json --reporters cli,htmlextra
k6	k6 run -e BASE_URL=... script.js | --vus 1 --duration 30s
ZAP	zap-baseline.py -t URL -r laporan.html | zap-api-scan.py -t SKEMA -f openapi | zap-full-scan.py -t URL
Keamanan statis	bandit -r . -ll | pip-audit | gitleaks detect

D. Glosarium

Istilah	Arti
AAA	Arrange-Act-Assert, pola menulis test
Boundary value	Menguji nilai tepat di batas aturan
Coverage	Persentase kode yang dijalankan oleh test
Factory / Fixture	Pembuat data uji / persiapan sebelum test
Flaky test	Test yang kadang lulus kadang gagal tanpa perubahan kode
IDOR	Mengakses data orang lain dengan mengubah ID di URL/permintaan
Mock / Fake	Pengganti layanan luar agar test cepat dan deterministik
N+1 query	Pola query berulang per baris yang memperlambat daftar/laporan
p95	Waktu respons yang dilampaui hanya oleh 5% permintaan terlambat
Page Object	Kelas yang membungkus elemen & aksi satu halaman di E2E
Regresi	Fitur lama rusak akibat perubahan baru
Staging	Lingkungan uji yang menyerupai produksi, berisi data palsu
UAT	User Acceptance Testing, penerimaan oleh klien/pengguna
VU	Virtual User, pengguna simulasi di k6

Selamat belajar. Mulailah dari satu test yang gagal, lalu buat hijau. Itulah kebiasaan yang membedakan programmer biasa dari programmer yang sistemnya bisa dipercaya untuk data anak.